UD 5. Interacción con el usuario: Eventos y Formularios

Índice

5.1 Listado de gestión de eventos 3
5.2. Manejadores de Eventos 5
5.2.1. Propagación de Eventos 7
5.2.2. Ejercicios 8
5.3. Modelos de eventos 9
5.3.1. Modelo de registro de eventos en linea 9
5.3.2. Modelo de registro de eventos tradicional 10
5.3.3. Modelo de registro avanzado de eventos según W3C 11
5.3.4. Orden de disparo de los eventos 12
5.4 Formularios 13
5.4.1. Campo de texto 13
5.4.2. Opciones de una lista 13
5.4.3. Checkbox 14
5.4.4. RadioButton 14
5.4.5. Ejercicios 15
5.5 Validación de formularios 16
5.5.1. Validar un campo de texto obligatorio 17
5.5.2. Validar un campo de texto con valores numéricos 18
5.5.3.Validar que se ha seleccionado una opción de una lista 18
5.5.4. Validar una dirección de email 19
5.5.5. Validar una fecha 19
5.5.6. Validar un número de DNI 20
5.5.7. Validar un número de teléfono 20
5.5.8. Validar que un checkbox ha sido seleccionado 21
5.5.9. Validar que un radiobutton ha sido seleccionado 22
5.5.10. Ejercicios 23
5.6. Expresiones Regulares 26
5.7. Utilización de «cookies» 27
5.8. Ejercicios de Ampliación 29

Autor: Roberto Marín

Módulo: Desarrollo Web Entorno Cliente (DAW2)

Año: 2024/2025

COPYRIGHT:

Reconocimiento-NoComercial-CompartirIgual. CC BY-NC-SA
Esta licencia permite a otros entremezclar, ajustar y construir a partir de su obra con fines no comerciales, siempre y cuando le reconozcan la autoría y sus nuevas creaciones estén bajo una licencia con los mismos términos.

No se permite el uso de este material en ninguna convocatoria oficial de oposiciones.

5.1 Listado de gestión de eventos

En este apartado vamos a responder a las siguientes preguntas: ¿Qué es un evento? ¿Qué tipos de eventos hay? ¿Cómo y cuando se ejecutan?

Según el W3schools «HTML events are «things» that happen to HTML elements.» es decir, son las acciones que se ejecutan sobre los elementos HTML.

Los eventos pueden ocurrir por acciones del navegador o del usuario. Cada uno de los elementos HTML tiene asociado una serie de eventos asociados.

Los manejadores nos van a permitir cómo ejecutar el evento y los escuchadores nos van a permitir lanzar o ejecutar el evento.

Además recordar que si no hay eventos, prácticamente no hay scripts. Teniendo en cuenta que en la mayoría de aplicaciones Web implementadas con JavaScript, suele haber eventos programados que disparan la ejecución de dichos scripts. Su explicación radica en que **JavaScript fue diseñado para añadir interactividad a las páginas: el usuario realiza algo y la página reacciona**.

Por este motivo, JavaScript necesita:

- ✓ Detectar las acciones del usuario para saber cuándo reaccionar
- ✓ Saber las funciones que queremos que ejecute cuando se produzcan esas acciones.

Cuando el usuario hace algo se produce un evento. También habrá algunos eventos que no están relacionados directamente con acciones de usuario: por ejemplo el evento de carga (load) de un documento, que se producirá automáticamente cuando un documento ha sido cargado.

La lista completa de eventos puede ser consultada: https://www.w3schools.com/jsref/dom_obj_event.asp

https://www.w3.org/TR/uievents

Evento Mouse	Detalle del evento
.onclick	Se ejecuta al hacer click con el ratón sobre un elemento
.ondblclick	Se ejecuta al hacer doble click con el ratón sobre un elemento
.onmousedown	Se ejecuta al presionar el ratón sobre un elemento
.onmouseenter	Se ejecuta al entrar con el ratón sobre un elemento
.onmouseover	Se ejecuta al entrar con el ratón sobre un elemento y sus secundarios
.onmouseleave	Se ejecuta al sacar el ratón de un elemento
.onmouseout	Se ejecuta al sacar el ratón de un elemento y sus secundarios
.onmousemove	Se ejecuta al mover el ratón sobre un elemento
.onmouseup	Se ejecuta al soltar el ratón sobre un elemento

Diferenciar entre onclick y onmouseup: Con onmouseup, se puede presionar el ratón fuera del elemento llevarlo dentro y en ese momento soltar el ratón.

Similar en cuanto al concepto relativo al Objeto nativo Screen visto en la UD3:

Propiedades Evento Mouse	**Detalle de las propiedades**
.clientX	Devuelve la coordenada horizontal del ratón, relativa a la VENTANA.
.clientY	Devuelve la coordenada vertical del ratón, relativa a la VENTANA.
.pageX	Devuelve la coordenada horizontal del ratón, relativa al DOCUMENTO.
.pageY	Devuelve la coordenada vertical del ratón, relativa al DOCUMENTO.
.screenX	Devuelve la coordenada horizontal del ratón, relativa a la PANTALLA.
.screenY	Devuelve la coordenada vertical del ratón, relativa a la PANTALLA.

5.2. Manejadores de Eventos

Los handlers (manejadores en traducción libre) son las funciones que se asocian a cada uno de los eventos. Los listeners son los eventos que lanzan la ejecución. La forma de definirlos es flexible y podremos definirlos según 5 grupos principales:

1. Integrados en el código HTML
2. Integrados con funciones externas
3. Manejadores semánticos
4. Mediante el uso de addEventListener
5. Mediante setAtribute

1. Integrados en el código HTML:

```
<p onclick="alert('me has pulsado');">Púlsame</p>
```

Esta es la forma más sencilla y menos profesional. Tiene el inconveniente de mezclar código JavaScript con HTML dificultando la depuración y la separación entre el diseño y la lógica de programación. Además, resulta tedioso conforme las acciones se complican.

Derivado de este método, podemos usar llamadas a funciones que se encuentran en otras partes del documento HTML o directamente en archivos .js externos.

2. Integrados en funciones externas:

```
function alerta(n){
alert("pulsada alerta " + n);
}

<p onclick="alerta('roja');">Púlsame para alerta roja</p>;
<p onclick="alerta('amarilla');">Púlsame para alerta amarilla</p>;
```

3. Manejadores semánticos

Para usar los manejadores semánticos navegaremos por el DOM hasta llegar al elemento que nos interesa. Para ello podremos hacerlo bien por su id o con cualquiera de los métodos de búsqueda (getElementByID, getElementByClass…) y después añadiremos el evento así:

```
document.getElementById('id_elemento').onclick = funcionClick;
```

Deberemos fijarnos en no terminar el nombre de la función con () ya que entonces se ejecutaría la función y si hubiera un return se asignaría al onclick del elemento, y todo ello sin ningún error de ejecución.

4. Mediante el uso de addEventListener

Por otra parte podemos usar los manejadores de eventos del DOM con la ventaja de que podremos asociar varios handler al mismo evento. Su sintaxis es:

```
document.getElementById('id_elemento').addEventListener("click", funcionClick, false);
```

Ejemplo:

```
let btn8 = document.getElementById('boton8');
btn8.addEventListener("click",function()
{funcion8('Ejecutando...8');
},false);
```

5. Mediante setAtribute

El método setAttribute está indicado para establecer atributos genéricos a elementos HTML. Sin embargo no se recomienda su uso para asignar listeners o modificar hojas de estilo.

- ✓ Sin parámetros:

```
let btn10 = document.getElementById('boton10');
btn10.setAttribute("onclick","funcion10();");
```

- ✓ Y con parámetros

```
let btn12 = document.getElementById('boton12');
btn12.setAttribute("onclick","funcion12('Ejecutando la función 12');");
```

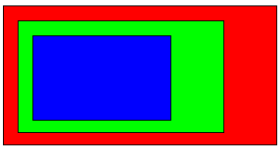
5.2.1. Propagación de Eventos

Cuando hay dos elementos superpuestos, y ambos tienen asociadas acciones para responder a un mismo evento, surge la duda de en qué orden se ejecutan las acciones.

Supongamos que tenemos tres elementos anidados de la forma siguiente:

```
<div style="width: 200px; height: 100px; background-color: red;"
          id="div1" onclick="alert('click on div1');">
    <div style="width: 150px; height: 80px; background: green; margin: 10px;"
          id="div2" onclick="alert('click on div2');">
      <div style="width: 100px; height: 60px; background: blue; margin: 10px;"
              id="div3" onclick="alert('click on div3'); return true;">
      </div>
    </div>
  </div>
```

El código HTML de arriba se visualiza como:



Si hacemos click con el ratón en el rectángulo azul, veremos que se presentan las alertas en secuencia:
click on div3
click on div2
click on div1

Es decir, que el evento se va propagando desde el elemento más interno hacia afuera, hasta alcanzar el elemento más externo. Esto es lo que se conoce como «bubbling». Si no se hubiera propagado, su ejecución sería: click on div1-click on div2-click on div3

Tener en cuenta que hay eventos con propagación y eventos sin propagación.
Diferencias entre el evento focusout y blur: El evento blur es disparado cuando un elemento ha perdido su foco y no se propaga. Mientras que el evento focusout además de perder el foco, se propaga (bubbling).

5.2.2. Ejercicios

Ejercicio 1: Realiza una aplicación relacionada con este módulo, que inicialmente esté cargada con una imagen y mediante un script, se le incorpore la funcionalidad de cambiar dicha imagen. Este proceso se realizará mediante el paso del ratón por encima del elemento. En el momento que se retira el ratón, la imagen cambiará por la inicial.
Nota: Aplicar el evento **onmouseover**, para reconocer la imagen por parte del ratón, Aplicar **onmouseout** para contemplar el caso en el que el ratón se encuentra fuera de la imagen.

Ejercicio 2: Coloca dos imágenes en pantalla: un coche y una moto. Crea un script para que al pasar el ratón por encima de las dos imágenes, las reconozca sacando por pantalla el texto "coche", "moto" o "no reconoce ninguna imagen" según proceda en cada caso.
Nota: Aplicando el evento **onmouseover**, para reconocer cada una de las imágenes, justo en el momento de pasar por encima el ratón. Emplear **onmouseout** para contemplar el caso en el que el ratón se encuentra fuera de ambas imágenes.

Ejercicio 3: Realizar una aplicación que al mover el ratón en cualquier punto de la ventana del navegador, se muestre la posición del puntero respecto del navegador y respecto de la página: Información que se muestra para los eventos del ratón. Para mostrar los mensajes, utilizar la función muestraInformacion().

Al pulsar cualquier tecla, el mensaje mostrado debe cambiar para indicar el nuevo evento y su información asociada: Información que se muestra para los eventos del teclado.

Añadir la siguiente característica al script: cuando se pulsa un botón del ratón, el color de fondo del cuadro de mensaje debe ser amarillo (#FFFFCC) y cuando se pulsa una tecla, el color de fondo debe ser azul (#CCE6FF). Al volver a mover el ratón, el color de fondo vuelve a ser blanco. El color de fondo del cuadro de información cambia en función del tipo de evento

Ejercicio 4: Crear un script que informe al usuario en que zona de la pantalla ha pulsado el ratón. Las zonas definidas son las siguientes: izquierda arriba, izquierda abajo, derecha arriba y derecha abajo.

5.3. Modelos de eventos

Antiguamente la creación de aplicaciones Web era mucho mas complejo debido a las incompatibilidades existentes entre navegadores. A pesar de que existían decenas de estándares para las diferentes tecnologías, los navegadores no los soportaban completamente o incluso los ignoraban.

Las principales incompatibilidades se producían en el lenguaje XHTML, en el soporte de hojas de estilos CSS y sobre todo, en la implementación de JavaScript. De todas ellas, la incompatibilidad más importante se daba precisamente en el modelo de eventos del navegador. Así, por lo tanto existían hasta tres modelos diferentes para manejar los eventos dependiendo del navegador en el que se ejecute la aplicación.

5.3.1. Modelo de registro de eventos en linea

En el modelo de registro de eventos en línea, el evento es añadido como un atributo más a la etiqueta HTML, como por ejemplo:

```
<a href="pagina.html" onClick="alert('Has pulsado en el enlace')">Pulsa aqui</a>
```

Cuando hacemos click en el enlace, se llama al gestor de eventos onClick (al hacer click) y se ejecuta el script; que contiene en este caso una alerta de JavaScript. También se podría realizar lo mismo pero llamando a una función:

```
<script>
const alertar = () => {
alert("Has pulsado en el enlace");
}
</script>
<a href="pagina.html" onClick="alertar()">Pulsa aqui</a>
```

No uses el modelo de registro de eventos en línea!.

Este modelo no se recomienda, y aunque lo has visto en ejemplos que hemos utilizado hasta ahora, tiene el problema de que estamos mezclando la estructura de la página web con la programación de la misma, y lo que se intenta hoy en día es separar la programación en JavaScript, de la estructura HTML, por lo que este modelo no nos sirve.

En el ejemplo anterior, cuando haces click en el enlace se mostrará la alerta y a continuación te conectará con la pagina.html. En ese momento desaparecerán de memoria los objetos que estaban en un principio, cuando se programó el evento. Esto puede ser un problema, ya que si por ejemplo la función a la que llamamos, cuando se produce el evento, tiene que realizar varias tareas, éstas tendrían que ser hechas antes de que nos conecte con la nueva página.

Éste modo de funcionamiento ha sido un principio muy importante en la gestión de eventos. Si un evento genera la ejecución de un script y además también se genera la acción por defecto para ese objeto entonces:

El script se ejecutará primero.
La acción por defecto se ejecutará después.
Evitar la acción por defecto

A veces es interesante el bloquear o evitar que se ejecute la acción por defecto. Por ejemplo, en nuestro caso anterior podríamos evitar que nos conecte con la nueva pagina.html. Cuando programamos un gestor de eventos, ese gestor podrá devolver un valor booleano true o false. Eso tendremos que programarlo con la instrucción return true|false. False quiere decir "no ejecutes la acción por defecto". Por lo tanto nuestro ejemplo quedará del siguiente modo:

```
<a href="pagina.html" onClick="alertar(); return false">Pulsa aqui</a>
```

De esa forma, cada vez que pulsemos en el enlace realizará la llamada a la función alertar() y cuando termine ejecutará la instrucción "return false", que le indicará al navegador que no ejecute la acción por defecto asignada a ese objeto (en este caso la acción por defecto de un hiperenlace es conectarnos con la página href de destino).

También se puede utilizar el método preventDefault() del evento para lo mismo.

5.3.2. Modelo de registro de eventos tradicional

En los navegadores antiguos, el modelo que se utilizaba era el modelo en línea. Con la llegada de DHTML, el modelo se extendió para ser más flexible. En este nuevo modelo el evento pasa a ser una propiedad del elemento, así que por ejemplo los navegadores modernos ya aceptan el siguiente código de JavaScript:

```
elemento.onclick = hacerAlgo;  // cuando el usuario haga click en el objeto, se llamará a
la función hacerAlgo()
```

Esta forma de registro, no fue estandarizada por el W3C, pero debido a que fue ampliamente utilizada por Netscape y Microsoft, todavía es válida hoy en día. La ventaja de este modelo es que podremos asignar un evento a un objeto desde JavaScript, con lo que ya estamos separando el código de la estructura. Fíjate que aquí los nombres de los eventos si que van siempre en minúsculas.

Para eliminar un gestor de eventos de un elemento u objeto, le asignaremos null:

```
elemento.onclick = null;
```

Otra gran ventaja es que, como el gestor de eventos es una función, podremos realizar una llamada directa a ese gestor, con lo que estamos disparando el evento de forma manual. Por ejemplo:

```
elemento.onclick();    // Estamos disparando el evento click de forma manual y se
ejecutará la función hacerAlgo()
```

Sin paréntesis

Fíjate que en el registro del evento no usamos paréntesis (). El método onclick espera que se le asigne una función completa. Si haces: element.onclick = hacerAlgo(); la

función será ejecutada y el resultado que devuelve esa función será asignado a onclick. Pero esto no es lo que queremos que haga, queremos que se ejecute la función cuando se dispare el evento.

5.3.3. Modelo de registro avanzado de eventos según W3C

El W3C en la especificación del DOM de nivel 2, pone especial atención en los problemas del modelo tradicional de registro de eventos. En este caso ofrece una manera sencilla de registrar los eventos que queramos, sobre un objeto determinado.
La clave para poder hacer todo eso está en el método addEventListener().

Este método tiene tres argumentos: el tipo de evento, la función a ejecutar y un valor booleano (true o false), que se utiliza para indicar cuándo se debe capturar el evento: en la fase de captura (true) o de burbujeo (false).

```
elemento.addEventListener('evento', función, false|true);
```

Por ejemplo para registrar la función alertar() de los ejemplos anteriores, haríamos:

```
document.getElementById("miEnlace").addEventListener('click', alertar, false);
const alertar = () => {
alert(`Te conectaremos con la página: ${this.href}`);
}
```

La ventaja de este método, es que podemos añadir tantos eventos como queramos:

```
document.getElementById("miEnlace").addEventListener('click', alertar, false);
document.getElementById("miEnlace").addEventListener('click', avisar, false);
document.getElementById("miEnlace").addEventListener('click', chequear, false);
```

Por lo tanto, cuando hagamos click en miEnlace se disparará la llamada a las tres funciones. Por cierto, el W3C no indica el orden de disparo, por lo que no sabemos cual de las tres funciones se ejecutará primero. Fíjate también, que el nombre de los eventos al usar addEventListener no lleva 'on' al comienzo.

También se pueden usar funciones anónimas (sin nombre de función), haciendo uso de la función flecha como ya venimos haciendo:

```
element.addEventListener('click', () => {
this.style.backgroundColor = '#cc0000';
}, false);
```

Uso de la palabra reservada **this**:
La palabra reservada this, tiene exactamente la misma funcionalidad que hemos visto en el modelo tradicional.

¿Qué eventos han sido registrados?
Uno de los problemas de la implementación del modelo de registro del W3C, es que no podemos saber con antelación, los eventos que hemos registrado a un elemento.
En el modelo tradicional si hacemos: alert(elemento.onclick), nos devuelve undefined, si no hay funciones registradas para ese evento, o bien el nombre de la función que hemos registrado para ese evento. Pero en este modelo no podemos hacer eso.
El W3C en el reciente nivel 3 del DOM, introdujo un método llamado eventListenerList, que almacena una lista de las funciones que han sido registradas a un elemento.
Para eliminar un evento de un elemento, usaremos el método removeEventListener():

```
elemento.removeEventListener('evento', función, false|true);
```

Para cancelar un evento, este modelo nos proporciona el método preventDefault().

5.3.4. Orden de disparo de los eventos

Imagina que tenemos un elemento contenido dentro de otro elemento, y que tenemos programado el mismo tipo de evento para los dos (por ejemplo el evento click). ¿Cuál de ellos se disparará primero? Sorprendentemente, ésto va a depender del tipo de navegador que tengamos.

El problema es muy simple. Imagina de nuevo las siguientes cajas:

Con una jerarquía de tres elementos y todos tienen programado el evento de click. Si el usuario hace click en el elemento2, provocará un click en Elemento1 y Elemento2.

¿Pero cuál de ellos se disparará primero?, ¿cuál es el orden de los eventos?

Modelo W3C

W3C decidió que, cuando se produce un evento en su modelo de eventos, primero se producirá la fase de captura hasta llegar al elemento de destino, y luego se producirá la fase de burbujeo hacia arriba. Este modelo es el estándar, que todos los navegadores deberían seguir para ser compatibles entre sí.

Tú podrás decidir cuando quieres que se registre el evento: en la fase de captura o en la fase de burbujeo. El tercer parámetro de addEventListener te permitirá indicar si lo haces en la fase de captura (true, ir del elemento más externo al más interno), o en la fase de burbujeo (false, ir del elemento más interno al más externo).

Por ejemplo:

```
elemento1.addEventListener('click', hacerAlgo1 ,true);
elemento2.addEventListener('click', hacerAlgo2, false);
```

Diagrama con el orden de burbujeo de los eventos entre elementos. Si el usuario hace click en el elemento2 ocurrirá lo siguiente:

El evento de click comenzará en la fase de captura. El evento comprueba si hay algún ancestro del elemento2 que tenga un evento de onclick para la fase de captura (true).

El evento encuentra un elemento1.hacerAlgo1() que ejecutará primero, pues está programado a true.

El evento viajará hacia el destino, pero no encontrará más eventos para la fase de captura. Entonces el evento pasa a la fase de burbujeo, y ejecuta hacerAlgo2(), el cuál hemos registrado para la fase de burbujeo (false).

El evento viaja hacia arriba de nuevo y chequea si algún ancestro tiene programado un evento para la fase de burbujeo. Éste no será el caso, por lo que no hará nada más.

Para detener la propagación del evento en la fase de burbujeo, disponemos del método stopPropagation(). En la fase de captura es imposible detener la propagación.

5.4 Formularios

Se van a detallar los diferentes elementos que están compuestos los formularios. Para la validación de los diferentes campos, ir a la Unidad Didáctica anterior.

Básicamente para acceder a los formularios tenemos dos maneras:

- Con los métodos de acceso directo a los nodos (getElementById(), etc…).
- Con document.forms y usando su identificador.

```
<body>
<form id="form1">
<input type="text" id="nombre" onChange="mostrar();">
</form>
</body>

<script type="text/javascript">
function mostrar(){
let f=document.forms["form1"];
let nom=f.elements["nombre"].value;
alert(nom);
}
</script>
```

5.4.1. Campo de texto

El valor de los elementos tipo text y textarea se obtiene según: elemento.value.

```
<body>
<form name="f2">
<input type="text" id="txt1">
<textarea id="txtarea1"></textarea>
</form>
</body>
<script type="text/javascript">
let txt = document.getElementById("txt1").value;
let txtArea = document.getElementById("txtarea1").value;
</script>
```

5.4.2. Opciones de una lista

```
<body>
<form name="f3">
  <select id="sel1" name="sel1">
     <option value="dom">Dom</option>
     <option value="objetos">Objetos</option>
     <option value="eventos">Eventos</option>
     <option value="formularios">Formularios</option>
  </select>
</form>
</body>
<script type="text/javascript">
window.onload = function() {
  asignarSelect();}
function asignarSelect(){
  let s= document.getElementById("sel1"); //el evento se asigna sobre el <select>.
  s.addEventListener("change",mostrarSelect,false);}
function mostrarSelect(){
  i=this.selectedIndex; console.log("Indice seleccionado: "+i);
  v=this[i].value; console.log("El value seleccionado es: "+v);
  h=this[i].innerHTML; console.log("El texto seleccionado es: "+h)}
</script>
```

5.4.3. Checkbox

Los elementos checkbox son similares a los radio con la diferencia de que son independientes unos de otros. Por tanto se evaluarán de manera independiente.

```
<form name="f2" id="f2">
 <label><input type="checkbox" value="coche" name="coche" id="coche">coche</label>
 <label><input type="checkbox" value="moto" name="moto" id="moto">moto</label>
 <label><input type="checkbox" value="camion" name="camion" id="camion">camion</label>
 <label><input type="checkbox" value="autobus" name="autobus" id="autobus">autobus</label>
</form>
```

La primera opción para acceder a ellos seria asignar un evento 1 a 1.

function asignarCheck(){
//podemos asignar directamente de 1 en 1 por su id.
document.getElementById("coche").addEventListener("change",mostrarCheck,false);
}
Sin embargo si el número de checkbox es muy grande o se generan en tiempo de ejecución puede resultar más interesante una asignación dinámica.

Usaremos la propiedad checked para comprobar si el elemento ha sido marcado y el evento onchange para lanzar la función.

```
function asignarCheck(){
  //Se capturan todos los checkbox
        let f=document.getElementById("f2");
        let ch=f.getElementsByTagName("input");
        for (var i=0; i<ch.length; i++){
             if (ch[i].type=="checkbox"){
                  ch[i].addEventListener("change",mostrarCheck,false);}}}
function mostrarCheck(ev){
  //la función es genérica ya que usamos la captura del evento para el mensaje.
         if (ev.target.checked){
               console.log(ev.target.name + " marcado");}
         else{
               console.log(ev.target.name + " desmarcado");}}
```

5.4.4. RadioButton

Los elementos radiobutton se caracterizan por ser mutuamente excluyentes y compartir el valor del atributo name. Deberemos seleccionarlos por nombre y recorrerlos para saber cuál es el marcado por el usuario.

```
<form name="f1">
  <input type="radio" name="r1" id="r1_1" value="si">SI
  <input type="radio" name="r1" id="r1_2" value="no">NO
  <input type="radio" name="r1" id="r1_3" value="a veces">A VECES
</form>
<script>
window.onload=function(){
asignarRadio()};
function asignarRadio(){
    //seleccionamos los elementos radio
    let r=document.getElementsByName("r1");
    //les asignamos un Listener, hay que hacerlo a todos.
    for (let i=0; i<r.length; i++){
            r[i].addEventListener("change",mostrarRadio,false);}}
```

```
function mostrarRadio(){
    console.log(this.value);}
</script>
```

5.4.5. Ejercicios

Ejercicio 1: Crea un formulario HTML con 4 campos de tipo text para introducir los 16 de números de una tarjeta de crédito. Debe comportarse de manera que tras pulsar el último número de cada input valide que es un número y pase el foco al siguiente campo input.

Ejercicio 2: En el formulario de la tarjeta de crédito añade un nuevo campo fecha de caducidad, un botón enviar que realice un submit y un manejador para el evento submit, que valide si la tarjeta se encuentra caducada. Al finalizar aparecerá un mensaje de "Tarjeta validada".

5.5 Validación de formularios

La principal utilidad de JavaScript en el manejo de los formularios es la validación de los datos introducidos por los usuarios. Antes de enviar un formulario al servidor, se recomienda validar mediante JS los datos introducidos por el usuario. De esta forma, si el usuario ha cometido algún error al rellenar el formulario, se le puede notificar de forma instantánea en el lado del cliente, sin necesidad de esperar respuesta del servidor.

Notificar los errores de forma inmediata mediante JavaScript mejora la satisfacción del usuario con la aplicación (lo que técnicamente se conoce como "mejorar la experiencia de usuario") y ayuda a reducir la carga de procesamiento en el servidor. (*Diseño de Interfaces Web: User Experience*)

Normalmente, la validación de un formulario consiste en llamar a una función de validación cuando el usuario pulsa sobre el botón de envío del formulario. En esta función, se comprueban si los valores que ha introducido el usuario cumplen las restricciones impuestas por la aplicación.

Aunque existen tantas posibles comprobaciones como elementos de formulario diferentes, algunas comprobaciones son muy habituales:

- Que se rellene un campo obligatorio
- Que se seleccione el valor de una lista desplegable
- Que la dirección de email indicada sea correcta
- Que la fecha introducida sea lógica
- Que se haya introducido un número donde se requiera un campo numérico.
- Etc.

A continuación se muestra el código JavaScript básico necesario para incorporar la validación a un formulario:

```
<form action="" method="" id="" name="" onsubmit="return validacion()">
   ...
</form>
```

Y el esquema de la función validacion() es el siguiente:

```
function validacion() {
        if (condicion que debe cumplir el primer campo del formulario) {
          // Si no se cumple la condicion...
          alert('[ERROR] El campo debe tener un valor de...');
          return false;
        }
        else if (condicion que debe cumplir el segundo campo del formulario) {
          // Si no se cumple la condicion...
          alert('[ERROR] El campo debe tener un valor de...');
          return false;
        }
        ...
        else if (condicion que debe cumplir el último campo del formulario) {
```

```
            // Si no se cumple la condicion...
            alert('[ERROR] El campo debe tener un valor de...');
            return false;
          }
          // Si el script ha llegado a este punto, todas las condiciones
          // se han cumplido, por lo que se devuelve el valor true
          return true;
}
```

El funcionamiento de esta técnica de validación se basa en el comportamiento del evento onsubmit de JavaScript. Al igual que otros eventos como onclick y onkeypress, el evento 'onsubmit varía su comportamiento en función del valor que se devuelve.

Así, **si el evento onsubmit devuelve el valor true, el formulario se envía como lo haría normalmente. Sin embargo, si el evento onsubmit devuelve el valor false, el formulario no se envía.** La clave de esta técnica consiste en comprobar todos y cada uno de los elementos del formulario. En cuando se encuentra un elemento incorrecto, se devuelve el valor false. Si no se encuentra ningún error, se devuelve el valor true.

Por lo tanto, en primer lugar se define el evento onsubmit del formulario como:

onsubmit="return validacion()"

Como el código JavaScript devuelve el valor resultante de la función validacion(), el formulario solamente se enviará al servidor si esa función devuelve true. En el caso de que la función validacion() devuelva false, el formulario permanecerá sin enviarse. Si se llega al final de la función, será que todas las condiciones se han cumplido, por lo que devuelve true y el formulario se envíará.

La notificación de los errores cometidos depende del diseño de cada aplicación. En el código del ejemplo anterior simplemente se muestran mensajes mediante la función alert() indicando el error producido. **Las Aplicaciones Web mejor diseñadas muestran cada mensaje de error al lado del elemento de formulario correspondiente y también suelen mostrar un mensaje principal indicando que el formulario contiene errores.**

Una vez definido el esquema de la función validacion(), se debe añadir a esta función el código correspondiente a todas las comprobaciones que se realizan sobre los elementos del formulario. A continuación, se muestran algunas de las validaciones más habituales de los campos de formulario.

5.5.1. Validar un campo de texto obligatorio

Se trata de forzar al usuario a introducir un valor en un cuadro de texto o textarea en los que sea obligatorio. La condición en JavaScript se puede indicar como:

```
valor = document.getElementById("campo").value;
if( valor == null || valor.length == 0 || /^\s+$/.test(valor) ) {
return false;
}
```

Para que se dé por completado un campo de texto obligatorio, se comprueba que:

- El valor introducido sea válido

- El número de caracteres introducido sea mayor que cero
- Que no se hayan introducido sólo espacios en blanco.

La primera parte de la condición, con **null** se revisa que no haya "ningún valor" de tipo objeto, array, numérico, cadena de texto o booleano.

La segunda parte de la condición obliga a que el texto introducido no tenga una longitud de cero caracteres, esto es, **que no sea un texto vacío**.

Por último, la tercera parte de la condición (/^\s+$/.test(valor)) obliga a que el valor introducido por el usuario no sólo esté formado por **espacios en blanco**. Esta comprobación se basa en el uso de "expresiones regulares", un recurso habitual en cualquier lenguaje de programación pero que por su complejidad no se van a estudiar. Por lo tanto, sólo es necesario copiar literalmente esta condición, poniendo especial cuidado en no modificar ningún carácter de la expresión.

5.5.2. Validar un campo de texto con valores numéricos

Se trata de obligar al usuario a introducir un valor numérico en un cuadro de texto. La condición JavaScript consiste en:

```
valor = document.getElementById("campo").value;
if(isNaN(valor)) {
return false;
}
```

Si el contenido de la variable valor no es un número válido, no se cumple la condición. La ventaja de utilizar la función interna isNaN() es que simplifica las comprobaciones, ya que JavaScript se encarga de tener en cuenta los decimales, signos, etc.

¡Recordatorio! Como ya sabéis, se muestran algunos resultados de la función isNaN():

```
isNaN(3);           // false
isNaN("3");         // false
isNaN(3.3545);      // false
isNaN(32323.345);   // false
isNaN(+23.2);       // false
isNaN("-23.2");     // false
isNaN("23a");       // true
isNaN("23.43.54"); // true
```

5.5.3.Validar que se ha seleccionado una opción de una lista

Se trata de obligar al usuario a seleccionar un elemento de una lista desplegable. El siguiente código JavaScript permite conseguirlo:

```
<select id="opciones" name="opciones">
   <option value="">- Selecciona un valor -</option>
   <option value="1">Primer valor</option>
   <option value="2">Segundo valor</option>
   <option value="3">Tercer valor</option>
</select>
indice = document.getElementById("opciones").selectedIndex;
if( indice == null || indice == 0 ) {
return false;}
```

A partir de la propiedad selectedIndex, se comprueba si el índice de la opción seleccionada es válido y además es distinto de cero. La primera opción de la lista (-Selecciona un valor –) no es válida, por lo que no se permite el valor 0 para esta propiedad selectedIndex.

5.5.4. Validar una dirección de email

Se trata de obligar al usuario a introducir una dirección de email con un formato válido. Por tanto, se comprueba que la dirección parezca válida, ya que no se comprueba si se trata de una cuenta de correo real y operativa. La condición JavaScript consiste en:

```
valor = document.getElementById("campo").value;
if( !(/\w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)/.test(valor)) ) {
return false;
}
```

La comprobación se realiza nuevamente mediante las expresiones regulares, ya que las direcciones de correo electrónico válidas pueden ser muy diferentes. Por otra parte, como el estándar que define el formato de las direcciones de correo electrónico es muy complejo, la expresión regular anterior es una simplificación. Aunque esta regla valida la mayoría de direcciones de correo electrónico utilizadas por los usuarios, no soporta todos los diferentes formatos válidos de email.

5.5.5. Validar una fecha

Las fechas suelen ser los campos de formulario más complicados de validar por la multitud de formas diferentes en las que se pueden introducir. El siguiente código asume que de alguna forma se ha obtenido el año, el mes y el día introducidos por el usuario:

```
let ano = document.getElementById("ano").value;
let mes = document.getElementById("mes").value;
let dia = document.getElementById("dia").value;

valor = new Date(ano, mes, dia);
if( !isNaN(valor) ) {
return false;}
```

¡Recordatorio! Como ya sabéis, la función Date(ano, mes, dia) es una función interna de JavaScript que permite construir fechas a partir del año, el mes y el día de la fecha. Es muy importante tener en cuenta que el número de mes se indica de 0 a 11, siendo 0 el mes de Enero y 11 el mes de Diciembre. Los días del mes siguen una numeración diferente, ya que el mínimo permitido es 1 y el máximo 31.

La validación consiste en intentar construir una fecha con los datos proporcionados por el usuario. Si los datos del usuario no son correctos, la fecha no se puede construir correctamente y por tanto la validación del formulario no será correcta. (Se complementa con la Primera Evaluación de DWEC)

5.5.6. Validar un número de DNI

Se trata de comprobar que el número proporcionado por el usuario se corresponde con un número válido del DNI. Aunque para cada país o región los requisitos del documento de identidad de las personas pueden variar, a continuación se muestra un ejemplo genérico fácilmente adaptable. La validación no sólo debe comprobar que el número esté formado por ocho cifras y una letra, sino que también es necesario comprobar que la letra indicada es correcta para el número introducido:

```
valor = document.getElementById("campo").value;
var letras = ['T', 'R', 'W', 'A', 'G', 'M', 'Y', 'F', 'P', 'D', 'X', 'B', 'N', 'J', 'Z', '
S', 'Q', 'V', 'H', 'L', 'C', 'K', 'E', 'T'];

if( !(/^\d{8}[A-Z]$/.test(valor)) ) {
return false;}

if(valor.charAt(8) != letras[(valor.substring(0, 8))%23]) {
return false;}
```

La primera comprobación asegura que el formato del número introducido es el correcto, es decir, que está formado por 8 números seguidos y una letra. Si la letra está al principio de los números, la comprobación sería /^[A-Z]\d{8}$/. Si en vez de ocho números y una letra, se requieren diez números y dos letras, la comprobación sería /^\d{10}[A-Z]{2}$/ y así sucesivamente.

La segunda comprobación aplica el algoritmo de cálculo de la letra del DNI y la compara con la letra proporcionada por el usuario. El algoritmo de cada documento de identificación es diferente, por lo que esta parte de la validación se debe adaptar convenientemente.

5.5.7. Validar un número de teléfono

Los números de teléfono pueden ser indicados de formas muy diferentes: con prefijo nacional, con prefijo internacional, agrupado por pares, separando los números con guiones, etc.

El siguiente script considera que un número de teléfono está formado por nueve dígitos consecutivos y sin espacios ni guiones entre las cifras:

```
valor = document.getElementById("campo").value;
if( !(/^\d{9}$/.test(valor)) ) {
return false;}
```

Una vez más, la condición de JavaScript se basa en el uso de expresiones regulares, que comprueban si el valor indicado es una sucesión de nueve números consecutivos. A continuación se muestran otras expresiones regulares que se pueden utilizar para otros formatos de número de teléfono:

Número	Expresión regular	Formato
900900900	/^\d{9}$/	9 cifras seguidas
900-900-900	/^\d{3}-\d{3}-\d{3}$/	9 cifras agrupadas de 3 en 3 y separadas por guiones
900 900900	/^\d{3}\s\d{6}$/	9 cifras, las 3 primeras separadas por un espacio
900 90 09 00	/^\d{3}\s\d{2}\s\d{2}\s\d{2}$/	9 cifras, las 3 primeras separadas por un espacio, las siguientes agrupadas de 2 en 2
(900) 900900	/^\(\d{3}\)\s\d{6}$/	9 cifras, las 3 primeras encerradas por paréntesis y un espacio de separación respecto del resto
+34 900900900	/^\+\d{2,3}\s\d{9}$/	Prefijo internacional (+ seguido de 2 o 3 cifras), espacio en blanco y 9 cifras consecutivas

5.5.8. Validar que un checkbox ha sido seleccionado

Si un elemento de tipo checkbox se debe seleccionar de forma obligatoria, JavaScript permite comprobarlo de forma muy sencilla:

```
elemento = document.getElementById("campo");
if( !elemento.checked ) {
return false;}
```

Si se trata de comprobar que todos los checkbox del formulario han sido seleccionados, es más fácil utilizar un bucle:

```
formulario = document.getElementById("formulario");
for(let i=0; i<formulario.elements.length; i++) {
    let elemento = formulario.elements[i];
    if(elemento.type == "checkbox") {
       if(!elemento.checked) {
       return false;
       }
    }
}
```

5.5.9. Validar que un radiobutton ha sido seleccionado

Aunque se trata de un caso similar al de los checkbox, la validación de los radiobutton presenta una diferencia importante: en general, la comprobación que se realiza es que el usuario haya seleccionado algún radio button de los que forman un determinado grupo. Mediante JavaScript, es sencillo determinar si se ha seleccionado algún radiobutton de un grupo.

El siguiente ejemplo recorre todos los radiobutton que forman un grupo y comprueba elemento por elemento si ha sido seleccionado. Cuando se encuentra el primer radiobutton seleccionado, se sale del bucle y se indica que al menos uno ha sido seleccionado.

```
opciones = document.getElementsByName("opciones");
let seleccionado = false;
  for(let i=0; i<opciones.length; i++) {
     if(opciones[i].checked) {
     seleccionado = true;
     break;
     }
  }
if(!seleccionado) {
return false;}
```

5.5.10. Ejercicios

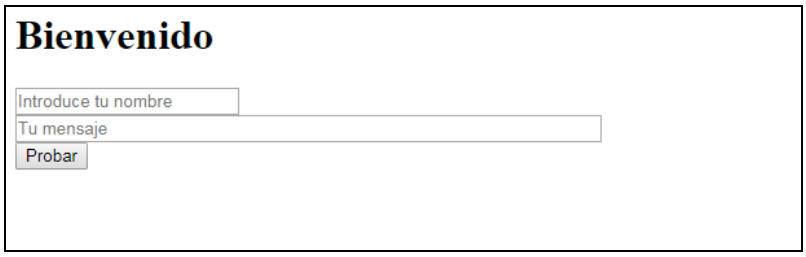
Ejercicio 1:

Crea un formulario con dos inputs “Nombre” y “Mensaje”. Crea un botón de validación para mandar un mensaje por correo electrónico. Sigue los pasos detallados anteriormente.

Ayuda:

```
<form action="mailto: ----@--.com" >
        Mensaje: <input type="text">
        Introduce tu nombre: <input type="text">
      <input value type="submit">
</form>
```

Ejercicio 2: Realiza el mismo ejercicio anterior y modifica la apariencia de los inputs, colocando interiormente el mensaje siguiente:



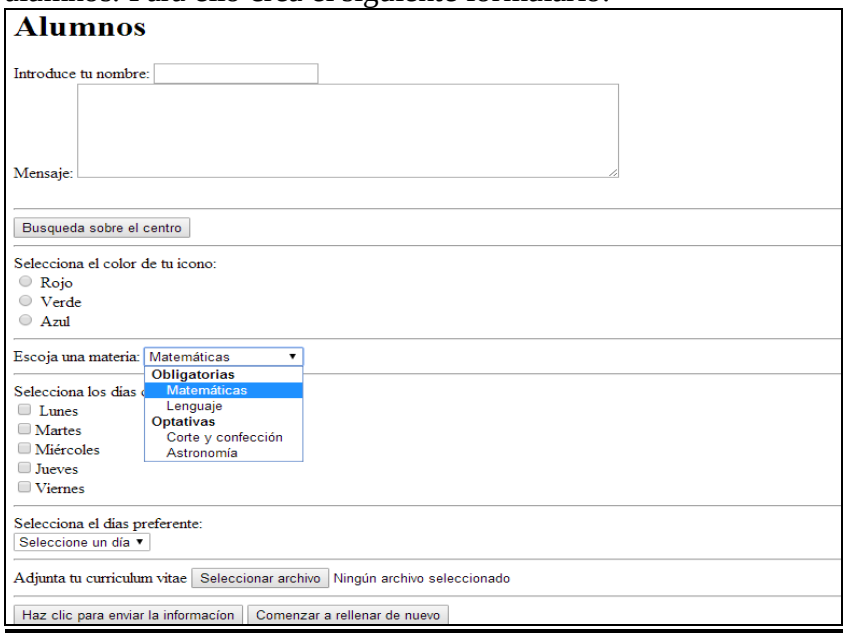
Ayuda: placeholder

Ejercicio 3: Validar los formularios 3, 7, 8, 9 y 10 de la UD0 Bloque 2. Conforme a los especificado en el 4.3

Ejercicio 4: Crea un formulario como aparece a continuación y envíalo por correo electrónico.

Ayuda: Trabaja con los inputs de tipo checkbox y radio. Crea dos botones adicionales de tipo submit y reset para enviar la información y borrar los datos introducidos.

Ejercicio 5: Se ha puesto en contacto el centro para facilitar la comunicación con los alumnos. Para ello crea el siguiente formulario:



Ayuda: crea un botón de tipo file para agregar archivos. Detalla los desplegables con select.

Ejercicio 6: Crea un formulario que se solicite el alta a un nuevo alumno.

Realice las siguientes validaciones: Lo grados solo tiene dos cursos (1º y 2º)
No se admite un año académico mayor que el actual, ni de 5 dígitos.
Todos los campos son obligatorios completarlo.

Ejemplo 7: Crea un formulario de subscripción de canal de prensa. Deben recogerse el nombre del usuario, NIF, correo electrónico y país de residencia.
Realice las siguientes validaciones:
El país de residencia debe ser España o Italia
El correo electrónico debe ser de tipo Hotmail o yahoo.es
El formato y número de Dni tiene que ser correcto
Todos los campos son obligatorios completarlo.

Ejemplo 8: Documento con dos formularios.
El primero con la información de alta del usuario: Nombre, Apellido, DNI, Email y Contraseña. Dichos datos se guardaran en la página del servidor alta.php.

En el segundo formulario los datos bancarios: 24 dígitos, CVV (de 3 dígitos) y Clave. Dichos datos se guardarán en la página del servidor pasarelaPago.php.

Nota:

- ☑ Todos los campos son obligatorios.
- ☑ Además de las validaciones de los campos Nombre, Apellido, emplea patrones para el DNI y el Email.
- ☑ La contraseña esta compuesta por un número, letra, una mayúscula, minúscula, carácter alfanumérico y tener 8 caracteres como mínimo.
- ☑ La clave serán 4 números.
- ☑ Tanto la contraseña como la clave no podrán visualizarse por pantalla (Type=password).

5.6. Expresiones Regulares

Las expresiones regulares son patrones que permiten buscar y/o reemplazar información. En JS las expresiones regulares son objetos. Las expresiones regulares están compuestas de patrones y banderas.

Patrones:
Se dispone de dos sintaxis:

```
let er = '/[0-9]/';
// o bien:
let er = new regExp('[0-9]');
```

Banderas:
Además del patrón, se emplean banderas o flags, existen 6 en JS:
i: Búsqueda case-insensitive.
g: Búsqueda global.
m: Modo multilínea de anclas ^ $, bandera "m").
s: Habilita el modo “dotall”, que permite que un punto . coincida con el carácter de línea nueva \n.
u: Permite soporte de Unicode. La bandera permite el procesamiento correcto de pares sustitutos. Más del tema en el capítulo Unicode: bandera "u" y clase \p{...}.
y: Modo “adhesivo”: búsqueda en la posición exacta del texto (cubierto en el capítulo Indicador adhesivo “y”, buscando en una posición.)

Ejemplo: (la i indica que la búsqueda no sea case sensitive)

```
let er = /[0-9]/i;
// o bien:
let er = new regExp('[0-9]',i);
```

Ampliar información:
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions

Métodos que usan expresiones regulares:
Las expresiones regulares se integran con los métodos de cadena. Ejemplo:
El método str.match(regex) busca todas las coincidencias de regex en la cadena str.

```
let str = "Estamos haciendo un arreglo, uno sobre un patron";
alert( str.match(/un/gi) );
//un,un,un
```

- ✓ exec: Método de RegExp. Ejecuta una búsqueda en un string. Devuelve un array de resultados o null si no hay coincidencias
- ✓ test: Método de RegExp. Ejecuta una búsqueda en un string. Devuelve true o false
- ✓ match: Método de String. Ejecuta una búsqueda en un string. Devuelve un array de resultados o null si no hay conincidencias
- ✓ search: Método de String. Ejecuta una búsqueda en un string. Devuelve el índice de la coincidencia o -1 si no hay conincidencias
- ✓ replace: Método de String. Ejecuta una búsqueda en un string y sustituye la coincidencia con un string de reemplazo
- ✓ split: Método de String. Usa la expresión regular para cortar el string en varios substrings

5.7. Utilización de «cookies»

Una cookie es realmente un string con pares de valores clave=valor separados por ;.

La gran diferencia con las estructuras de datos que hemos visto hasta ahora es que las cookies van a permanecer en el cliente (durante la vida de la cookie) incluso aunque se cierre el navegador (siempre y cuando el cliente no borre las cookies).

Tiene similitudes con LocalStorage (Ver Unidad Didáctica 3), aunque almacenamiento en cookies y en LocalStorage tienen distintos propósitos.

Diferencias entre LocalStorage y usar Cookies:

- ✓ Las cookies pueden ser directamente leídas desde el servidor, en cambio, el LocalStorage solo guarda y permite leer los datos desde el cliente.
- ✓ Las cookies tienen un límite más reducido de almacenamiento en comparación al LocalStorage y éste, técnicamente es más fácil de usar y te permite más con qué trabajar.

CONCLUSIÓN: Lo que determinaría si se debe o no usar un método u otro para almacenar y persistir los datos del usuario en el sitio web sería si se requiere de estos en el cliente o en el servidor.

Para ampliar información: https://cybmeta.com/que-son-las-cookies-y-como-funcionan

Ejemplo:

```
newCookie = "usuario=profesor"
document.cookie = newCookie;
alert(document.cookie);
newCookie = "usuario=profesor"
document.cookie = newCookie;
//descomentar la siguiente línea para ejecutar.
//alert(document.cookie);
```

Veremos cómo se ha añadido una cookie de par usuario=profesor y que si modificamos el valor de 'profesor' la cookie se modifica. Además si, en lugar de usar la key usuario, usamos otra, permanecen ambas cookies. Esto se debe a que document.cookie no es una propiedad si no un método que añade las cookies si no existen y las modifica en caso contrario.

El par <name>=<value> es el único requerido pero podemos ir definiendo otros que si no especificamos quedarán con sus valores por defecto.

NOTA: Dado que las cookies se envian en las cabeceras HTTP se deben codificar correctamente. Asi utilizaremos encodeURIComponent() y pondremos las fechas en formato UTC con Date.toUTCString():

Además del nombre de la cookie podremos añadir los siguientes parámetros opcionales:

- ✓ expires=. Indica hasta cuando dura la cookie. En combinación con max-age prevalece esta última.

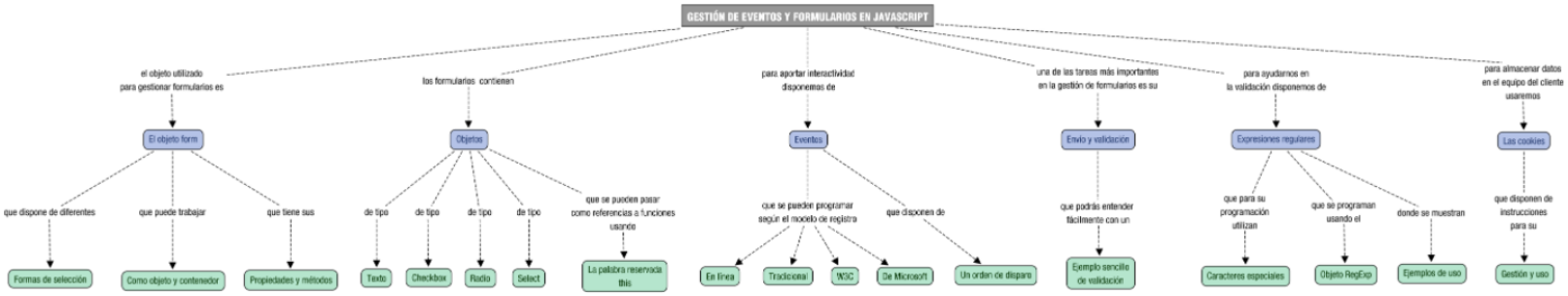
- ✓ max-age. Duración máxima de la cookie en segundos. Si no se establecen ninguno de los 2 atributos de duración, la cookie sólo será válida mientras dure la sesión.
- ✓ Path. Ruta para la cuál la cookie es válida. Si no se especifica nada será válida para el directorio actual.
- ✓ Domain. Dominio para el cuál la cookie es válida. Si no se especifica nada será válida para el subdominio actual. Por seguridad no se permite crear cookies para dominios diferentes al que crea la cookie.
- ✓ Secure. Sin valor. Si se indica sólo será válida para conexiones encriptadas (HTTPS por ejemplo).

Nota: Para modificar una cookie con domain o path especificados, estos deberán coincidir o se creará otra cookie diferente.

5.8. Ejercicios de Ampliación

Ejercicio 1: Realizar la validación del formulario facilitado en el enunciado, cumpliendo los siguientes requisitos:

- ✓ Programar el código de JavaScript en un fichero independiente. La única modificación que se permite realizar en el fichero .html es la de escribir la referencia al fichero de JavaScript.
- ✓ Almacenar en una cookie el número de intentos de envío del formulario que se van produciendo y mostrar un mensaje en el contenedor "intentos" similar a: "Intento de Envíos del formulario: X". Es decir cada vez que le demos al botón de enviar tendrá que incrementar el valor de la cookie en 1 y mostrar su contenido en el div antes mencionado. Nota: para poder actualizar el contenido de un contenedor o div la propiedad que tenemos que modificar para ese objeto es innerHTML.
- ✓ Cada vez que los campos NOMBRE y APELLIDOS pierdan el foco, el contenido que se haya escrito en esos campos se convertirá a mayúsculas.
- ✓ Realizar una función que valide los campos de texto NOMBRE y APELLIDOS. Si se produce algún error mostrar el mensaje en el contenedor "errores" y poner el foco en los campos correspondientes.
- ✓ Validar la EDAD que contenga solamente valores numéricos y que esté en el rango de 0 a 105. Si se produce algún error mostrar el mensaje en el contenedor "errores" y poner el foco en el campo EDAD.
- ✓ Validar el NIF. Utilizar una expresión regular que permita solamente 8 números un guión y una letra. Si se produce algún error mostrar el mensaje en el contenedor "errores" y poner el foco en el campo NIF. No es necesario validar que la letra sea correcta. Explicar las partes de la expresión regular mediante comentarios.
- ✓ Validar el E-MAIL. Utilizar una expresión regular que nos permita comprobar que el e-mail sigue un formato correcto. Si se produce algún error mostrar el mensaje en el contenedor "errores" y poner el foco en el campo E-MAIL. Explicar las partes de la expresión regular mediante comentarios.
- ✓ Validar que se haya seleccionado alguna de las PROVINCIAS. Si se produce algún error mostrar el mensaje en el contenedor "errores" y poner el foco en el campo PROVINCIA.
- ✓ Validar el campo FECHA utilizando una expresión regular. Debe cumplir alguno de los siguientes formatos: dd/mm/aaaa o dd-mm-aaaa. No se pide validar que sea una fecha de calendario correcta. Si se produce algún error mostrar el mensaje en el contenedor "errores" y poner el foco en el campo FECHA. Explicar las partes de la expresión regular mediante comentarios.
- ✓ Validar el campo TELEFONO utilizando una expresión regular. Debe permitir 9 dígitos obligatorios. Si se produce algún error mostrar el mensaje en el contenedor "errores" y poner el foco en el campo TELEFONO. Explicar las partes de la expresión regular mediante comentarios.
- ✓ Validar el campo HORA utilizando una expresión regular. Debe seguir el patrón de hh:mm. No es necesario validar que sea una hora correcta. Si se produce algún error mostrar el mensaje en el contenedor "errores" y poner el foco en el campo HORA. Explicar las partes de la expresión regular mediante comentarios.
- ✓ Pedir confirmación de envío del formulario. Si se confirma el envío realizará el envío de los datos; en otro caso cancelará el envío.


GESTIÓN DE EVENTOS Y FORMULARIOS EN JAVASCRIPT
el objeto utilizado para gestionar formularios es
El objeto form
que dispone de diferentes
Formas de selección
que puede trabajar
Como objeto y contenedor
que tiene sus
Propiedades y métodos
los formularios contienen
Objetos
de tipo
Texto
de tipo
Checkbox
de tipo
Radio
de tipo
Select
que se pueden pasar como referencias a funciones usando
La palabra reservada this
para aportar interactividad disponemos de
Eventos
que se pueden programar según el modelo de registra
En línea
Tradicional
W3C
De Microsoft
que disponen de
Un orden de disparo
una de las tareas más importantes en la gestión de formularios es su
Envío y validación
que podrás entender fácilmente con un
Ejemplo sencillo de validación
para ayudarnos en la validación disponemos de
Expresiones regulares
que para su programación utilizan
Caracteres especiales
que se programan usando el
Objeto RegExp
donde se muestran
Ejemplos de uso
para almacenar datos en el equipo del cliente usaremos
Las cookies
que disponen de instrucciones para su
Gestión y uso